What Makes Factoring So Hard?

A Peek into What We've Actually Factored With Shor's Algorithm

Jean (Shujia) Zhang

April 2026

Contents

1 Introduction to Shor's Algorithm

1.1 Why is Shor's Algorithm Important?

Shor's algorithm [1] is a quantum algorithm that dramatically reduces the complexity of integer factorization, achieving an exponential speedup over the best known classical methods. It holds a foundational place in the history of quantum computing as the first quantum algorithm with a clear and consequential practical application: the potential to break widely used cryptographic systems.

The security of modern cryptosystems, such as RSA, relies on the computational difficulty of factoring large integers. Given a number $N = pq$, where p and q are large primes, it is straightforward to verify that p and q divide N. However, recovering p and q from N alone is believed to be computationally intractable. This asymmetry—hard to solve, easy to verify—is what underpins the security of RSA and related schemes.

Shor's algorithm leverages quantum mechanical principles to render this problem efficiently solvable, reducing the runtime to polynomial complexity. If realized on a sufficiently large and reliable quantum computer, it would pose a serious threat to the security of current cryptographic infrastructure.

To date, however, experimental demonstrations remain limited. Small numbers such as 15 [2, 3] and 21 [4] have been "factored" using implementations inspired by Shor's algorithm, but whether these demonstrations constitute generic factoring circuits is debatable. In many cases, the implementations rely on compiled circuits, prior knowledge of the factors, or problem-specific optimizations. Similarly, claims of factoring larger numbers [5] often either do not employ Shor's algorithm in its full generality or incorporate shortcuts that undermine the validity of the result as true factorization.

This raises a natural question: what does it mean to build a generic factoring circuit on a quantum computer? This paper aims to clarify that question by examining the criteria for generic factorization, evaluating the capabilities and limitations of current experimental implementations, and proposing how arithmetic circuits could be designed to enable a base-agnostic and modulus-agnostic approach to quantum factorization.

1.2 Shor's Algorithm: An Overview

In order to understand how Shor's Algorithm achieves its speedup, it is helpful to first examine how factoring can be approached using classical number theory, and identify the computational bottleneck where quantum methods provide an advantage.

1.2.1 Factoring with Classical Number Theory

Given an integer $N = pq$, where p and q are primes, how can we recover p and q?

Step 1: Choose a random integer a.

Select a random integer a such that $1 < a < N$, and check whether it shares a nontrivial factor with N.

This can be done by computing $\gcd(a, N)$ using Euclid's algorithm.

If $\gcd(a, N) \neq 1$, then we have already found a nontrivial factor of N, and the algorithm terminates.

If $\gcd(a, N) = 1$, then a and N are coprime, and we proceed to the next step.

Step 2: Compute the order r of $a \bmod N$.

Since a and N are coprime, Euler's theorem guarantees the existence of a positive integer r such that

$$a^r \equiv 1 \pmod{N}.$$

The smallest such r is called the *order* of a modulo N.

This step is the computational bottleneck of the classical approach. To find r classically, one typically performs a brute-force search, testing successive values of x until the congruence $a^x \equiv 1 \pmod{N}$ is satisfied. In the worst case, r may be on the order of N, leading to exponential time complexity, $O(2^n)$, where n is the number of bits required to represent N.

This is precisely the step where quantum algorithms provide an exponential advantage, as will be discussed later.

Step 3: Use r to extract factors of N.

Once we have found r such that $a^r \equiv 1 \pmod{N}$, we can rewrite this as

$$a^r - 1 \equiv 0 \pmod{N}.$$

If r is even, we can factor the left-hand side using the difference of squares:

$$a^r - 1 = \left(a^{\frac{r}{2}} - 1\right)\left(a^{\frac{r}{2}} + 1\right).$$

Thus,

$$\left(a^{\frac{r}{2}} - 1\right)\left(a^{\frac{r}{2}} + 1\right) \equiv 0 \pmod{N},$$

which implies that N divides the product. Consequently, it is likely that $\left(a^{\frac{r}{2}} - 1\right)$ or $\left(a^{\frac{r}{2}} + 1\right)$ shares a nontrivial factor with N.

We can therefore compute

$$\gcd\left(a^{\frac{r}{2}} - 1, N\right) \quad \text{and} \quad \gcd\left(a^{\frac{r}{2}} + 1, N\right).$$

If either of these yields a nontrivial divisor, we have successfully factored N.

Failure conditions.

The procedure may fail in two cases:

- If r is odd, then $\frac{r}{2}$ is not an integer, and the above factorization cannot be applied.
- If $\gcd\left(a^{\frac{r}{2}} \pm 1, N\right) = 1$ or N, then no nontrivial factor is obtained.

In either case, the algorithm must be restarted with a new choice of a.

1.2.2 Shor's Algorithm Comes to the Rescue

In the classical approach described above, the computational bottleneck lies in the order-finding step, which requires exponential time, $O(2^n)$, as it involves testing values sequentially.

Shor's insight was to exploit two key features of quantum mechanics:

Quantum Superposition. Instead of evaluating a^x for each value of x individually, a quantum computer can prepare a superposition of all 2^n possible values of x using Hadamard gates. This allows the computation of $a^x \bmod N$ to be performed simultaneously across all inputs.

Periodicity of $a^x \bmod N$. The function $f(x) = a^x \bmod N$ is periodic with period r, the order of a modulo N. Since Fourier transforms are well-suited for extracting periodic structure, the Quantum Fourier Transform (QFT) can be used to efficiently determine r.

As a result, Shor's algorithm consists of two main components: a modular exponentiation stage, in which $a^x \bmod N$ is computed over a superposition of inputs, and a period-finding stage, which uses the QFT followed by a classical continued fractions algorithm to extract r.

Implementing modular exponentiation. To understand how to compute $a^x \bmod N$ in a quantum circuit, we first consider the simpler problem of computing a^x, where x is represented as an n-bit string $x_{n-1}x_{n-2}\cdots x_1x_0$. This bitstring represents the integer

$$x = \sum_{k=0}^{n-1} 2^k x_k.$$

Using this binary expansion, we can rewrite a^x as

$$a^x = a^{\sum_{k=0}^{n-1} 2^k x_k} = \prod_{k=0}^{n-1} a^{2^k x_k}.$$

This decomposition is crucial: it shows that a^x can be constructed as a sequence of conditional multiplications. For each bit x_k, we multiply by a^{2^k} if $x_k = 1$, and do nothing if $x_k = 0$. Thus, the computation can be implemented using controlled multiplication operations.

To extend this to modular exponentiation, we simply perform all operations modulo N. Instead of multiplying by $a^{2^k x_k}$, we multiply by $a^{2^k} \bmod N$, conditioned on the value of x_k. This is justified by the identity

$$(A \cdot B) \bmod N = \big((A \bmod N)(B \bmod N)\big) \bmod N,$$

which ensures that modular multiplication composes correctly.

Therefore, by chaining together controlled modular multiplication gates, we can efficiently construct a quantum circuit that computes $a^x \bmod N$.

Circuit interpretation. This construction is reflected in the standard circuit model of Shor's algorithm, shown in Figure 3. The controlled-U gates correspond to the modular multiplication operations described above, each implementing multiplication by $a^{2^k} \bmod N$, conditioned on the k-th qubit of the input register.

The input register is initialized in a uniform superposition using Hadamard gates, enabling the simultaneous evaluation of $a^x \bmod N$ for all x.

Notably, the number of qubits in the input register is typically chosen to be $2n$, where $n = \lceil \log_2 N \rceil$. This ensures that the subsequent Quantum Fourier Transform can resolve the period r with sufficiently high precision for successful recovery via the continued fractions algorithm.

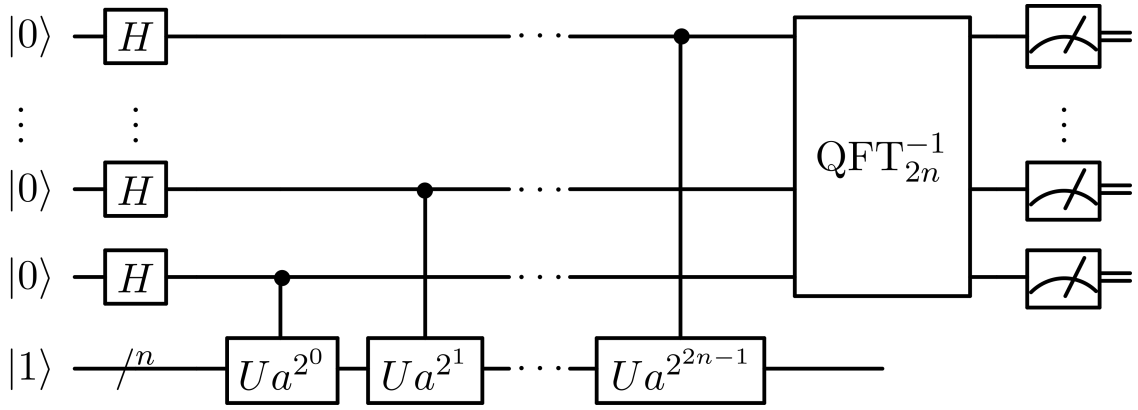

Figure 1: Circuit realization of Shor's algorithm

Quantum Fourier Transform

After the modular exponentiation stage, the quantum state takes the form

$$\sum_x |x\rangle \otimes |f(x)\rangle\,,$$

where $f(x) = a^x \bmod N$.

As illustrated in Figure 2, the function $f(x)$ is periodic with period r (in the example shown, $r = 4$). This periodicity is the key structure that Shor's algorithm exploits.

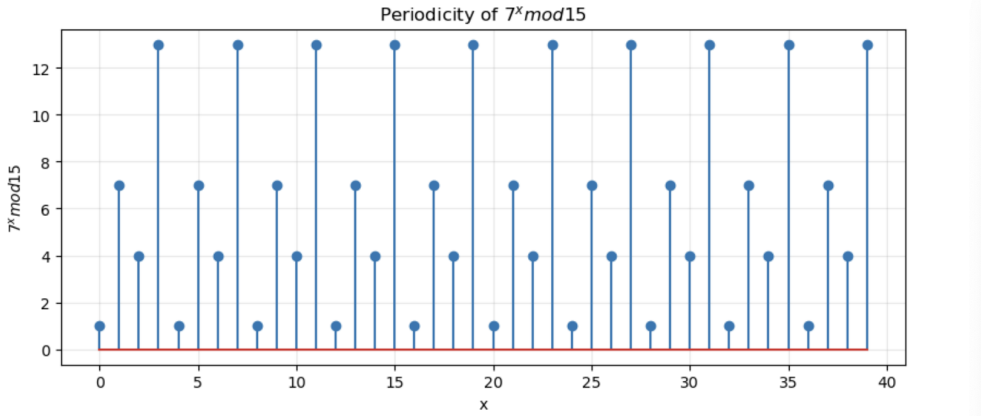

Figure 2: Periodicity of $a^x \bmod N$ for an example with $r = 4$

We can reorganize the superposition by grouping together values of x that yield the same function value. This gives

$$\sum_{k=0}^{r-1} \left(\sum_m |k + mr\rangle \right) \otimes |f(k)\rangle \,,$$

where k represents the residue class of x modulo r, i.e., its position within one period.

For example, when $r = 4$, the integers are partitioned into the following classes:

$$\begin{aligned} k = 0: &\quad 0, 4, 8, 12, \ldots \\ k = 1: &\quad 1, 5, 9, 13, \ldots \\ k = 2: &\quad 2, 6, 10, 14, \ldots \\ k = 3: &\quad 3, 7, 11, 15, \ldots \end{aligned}$$

Each class corresponds to a fixed value of $f(k)$.

We now apply the Quantum Fourier Transform (QFT) to the first register:

$$QFT\left(\sum_{k=0}^{r-1} \sum_m |k + mr\rangle \right) = \sum_{k=0}^{r-1} \sum_m QFT\left(|k + mr\rangle \right) \tag{1}$$

$$= \sum_{k=0}^{r-1} \sum_m \frac{1}{\sqrt{2^{2n}}} \sum_{y=0}^{2^{2n}-1} e^{\frac{2\pi i (k+mr) y}{2^{2n}}} |y\rangle \tag{2}$$

$$= \sum_{k=0}^{r-1} \frac{1}{\sqrt{2^{2n}}} \sum_{y=0}^{2^{2n}-1} e^{\frac{2\pi i k y}{2^{2n}}} \left(\sum_m e^{\frac{2\pi i m r y}{2^{2n}}} \right) |y\rangle \,. \tag{3}$$

The inner sum,

$$\sum_m e^{\frac{2\pi i m r y}{2^{2n}}},$$

constructively interferes when $\frac{ry}{2^{2n}}$ is close to an integer. As a result, the amplitude is sharply peaked at values of y satisfying

$$y \approx \frac{j \cdot 2^{2n}}{r}, \quad \text{for integers } j.$$

Therefore, after measurement, the observed values of y are most likely to be close to integer multiples of $\frac{2^{2n}}{r}$. By repeating the algorithm multiple times, we obtain several such samples in the frequency domain.

The remaining task is to recover r from a measured value of y. This can be accomplished using the classical method of continued fractions, which allows us to approximate the ratio $\frac{y}{2^{2n}}$ and extract the period r.

Continued Fractions

Given that the measurement outcome satisfies

$$y \approx \frac{j \cdot 2^{2n}}{r},$$

we can rewrite this as

$$\frac{y}{2^{2n}} \approx \frac{j}{r}.$$

From a single sample produced by the quantum computer, we do not know the values of j or r individually; we only know that they are integers whose ratio is well-approximated by $\frac{y}{2^{2n}}$.

To recover r from this approximation, we use the method of continued fractions.

A continued fraction is an expression of the form

$$x = b_0 + \cfrac{a_1}{b_1 + \cfrac{a_2}{b_2 + \cfrac{a_3}{b_3 + \cdots}}}.$$

By truncating this expansion at various depths, we obtain a sequence of rational approximations of the form $\frac{a}{b}$, where a and b are integers.

Applying this procedure to $\frac{y}{2^{2n}}$ allows us to recover a candidate fraction $\frac{j}{r}$, from which the period r can be extracted.

1.2.3 Toy Example with $a = 7$ and $N = 15$

The aim of this toy example is to illustrate how Shor's algorithm can be implemented for specific choices of a and N, as well as to demonstrate the optimization techniques that can be used to simplify the circuit. This provides a foundation for understanding the experimental setups of various demonstrations that claim to have factored 15 or 21. However, like those experiments, this example does not constitute a generic factoring circuit.

Step 1: Determining the Number of Qubits

The lower register, which stores the value $|a^x \bmod N\rangle$, must be large enough to represent integers from 0 to $N-1$. In this case, $N = 15$, so the register must represent values from 0 to 14. Since 14 in binary is 1110, we require $n = 4$ qubits for the lower register.

For the upper register, which stores the superposition of inputs x, we typically allocate $2n$ qubits to ensure sufficient precision for recovering the period r. Therefore, we use 8 qubits for the upper register.

With these choices, we can construct the corresponding quantum circuit (e.g., using Qiskit).

Step 2: Implementing Modular Exponentiation

Here is where circuit optimization comes in for specific values of a and N. As discussed in the previous section, modular exponentiation can be decomposed into a sequence of controlled modular multiplications.

Since the upper register contains 8 qubits, we must implement controlled modular multiplications corresponding to $a^{2^k} \bmod N$ for $k = 0, 1, \ldots, 7$. These values can be computed classically in advance.

From the figure, we observe that

$$7^{2^0} \equiv 7 \pmod{15}, \quad 7^{2^1} \equiv 4 \pmod{15}, \quad 7^{2^k} \equiv 1 \pmod{15} \text{ for } k \geq 2.$$

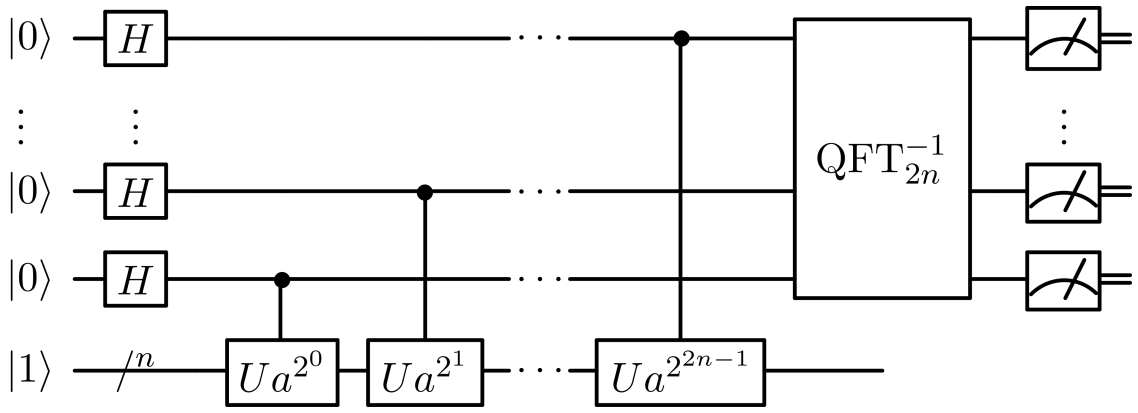

Figure 3: Circuit realization of Shor's algorithm

```
print([pow(7, 2**k, 15) for k in range(8)])
```

```
[7, 4, 1, 1, 1, 1, 1, 1]
```

Figure 4: Values of $a^{2^k} \bmod N$ for $a = 7$ and $N = 15$

This greatly simplifies the circuit: only the first two controlled operations have a nontrivial effect, while the remaining ones act as the identity.

Now, to implement controlled modular multiplication by 7 mod 15 and 4 mod 15, it is helpful to analyze how these operations act on the computational basis states, as summarized in Table 1.

$\|x\rangle$	$\|7x \bmod 15\rangle$
$\|0\rangle$	$\|0\rangle$
$\|1\rangle$	$\|7\rangle$
$\|2\rangle$	$\|14\rangle$
$\|3\rangle$	$\|6\rangle$
$\|4\rangle$	$\|13\rangle$
$\|5\rangle$	$\|5\rangle$
$\|6\rangle$	$\|12\rangle$
$\|7\rangle$	$\|4\rangle$
$\|8\rangle$	$\|11\rangle$
$\|9\rangle$	$\|3\rangle$
$\|10\rangle$	$\|10\rangle$
$\|11\rangle$	$\|2\rangle$
$\|12\rangle$	$\|9\rangle$
$\|13\rangle$	$\|1\rangle$
$\|14\rangle$	$\|8\rangle$

(a) Decimal representation

$\|x\rangle$	$\|7x \bmod 15\rangle$
$\|0000\rangle$	$\|0000\rangle$
$\|0001\rangle$	$\|0111\rangle$
$\|0010\rangle$	$\|1110\rangle$
$\|0011\rangle$	$\|0110\rangle$
$\|0100\rangle$	$\|1101\rangle$
$\|0101\rangle$	$\|0101\rangle$
$\|0110\rangle$	$\|1100\rangle$
$\|0111\rangle$	$\|0100\rangle$
$\|1000\rangle$	$\|1011\rangle$
$\|1001\rangle$	$\|0011\rangle$
$\|1010\rangle$	$\|1010\rangle$
$\|1011\rangle$	$\|0010\rangle$
$\|1100\rangle$	$\|1001\rangle$
$\|1101\rangle$	$\|0001\rangle$
$\|1110\rangle$	$\|1000\rangle$

(b) Binary representation

Table 1: Action of multiplication by 7 mod 15 on computational basis states

From Table 1, we observe that multiplication by 7 mod 15 acts as a permutation on the computational basis states. Therefore, it can be implemented as a permutation unitary, which can be decomposed into SWAP and NOT gates.

Circuit construction for $7x \bmod 15$**.** Explicitly, the transformation can be written as

$$|x_3x_2x_1x_0\rangle \mapsto |\bar{x}_0\bar{x}_3\bar{x}_2\bar{x}_1\rangle\,.$$

This operation can be implemented in two steps:

1. Perform a cyclic permutation of the qubits:

$$|x_3x_2x_1x_0\rangle \mapsto |x_0x_3x_2x_1\rangle\,.$$

2. Apply a bitwise NOT operation to all qubits:

$$|x_0x_3x_2x_1\rangle \mapsto |\bar{x}_0\bar{x}_3\bar{x}_2\bar{x}_1\rangle\,.$$

This yields the following circuit implementation:

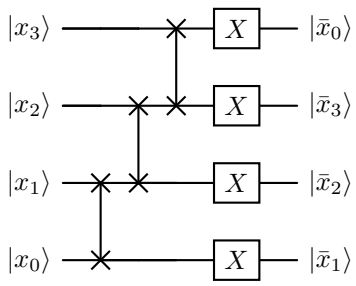

Figure 5: Implementation of $7x$ mod 15 using SWAP gates followed by bitwise NOT operations

Each SWAP gate can be further decomposed into CNOT gates, yielding a fully elementary gate decomposition:

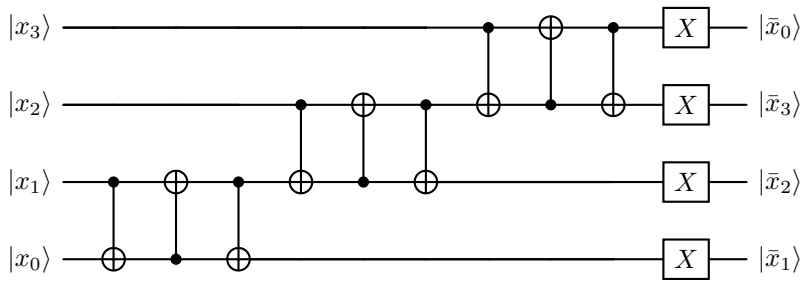

Figure 6: Decomposition of $7x$ mod 15 into CNOT and NOT gates

Note that this transformation maps $|0000\rangle$ to $|1111\rangle$. In practice, this does not pose a problem, as the state $|0000\rangle$ does not arise during the computation.

Controlled modular multiplication. To incorporate this modular multiplication operation into Shor's algorithm, we must implement a controlled version, where the transformation is applied only when a control qubit in the upper register is $|1\rangle$. This is shown in Figure 7. Using the same approach, we construct a controlled modular multiplication circuit for $4x$ mod 15 (Figure 8).

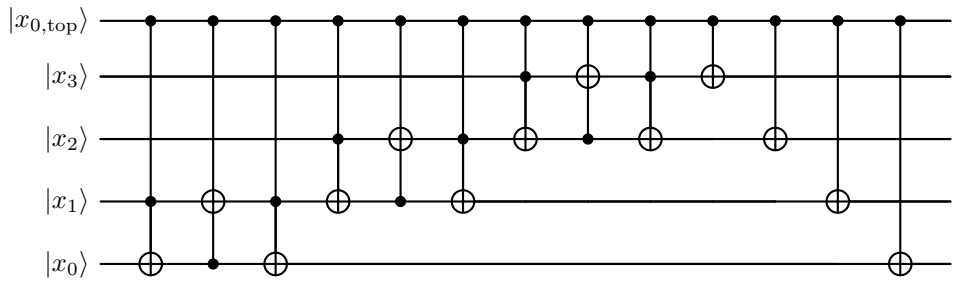

Figure 7: Controlled implementation of $7x \bmod 15$

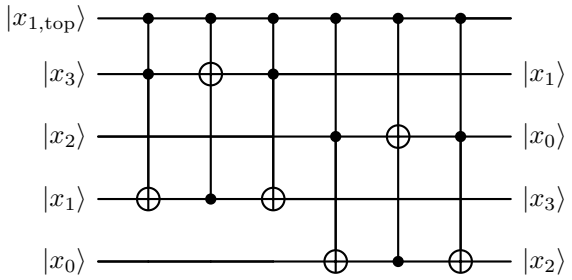

Figure 8: Controlled implementation of $4x \bmod 15$

Combined modular exponentiation circuit. Combining these components yields the full modular exponentiation circuit (Figure 9):

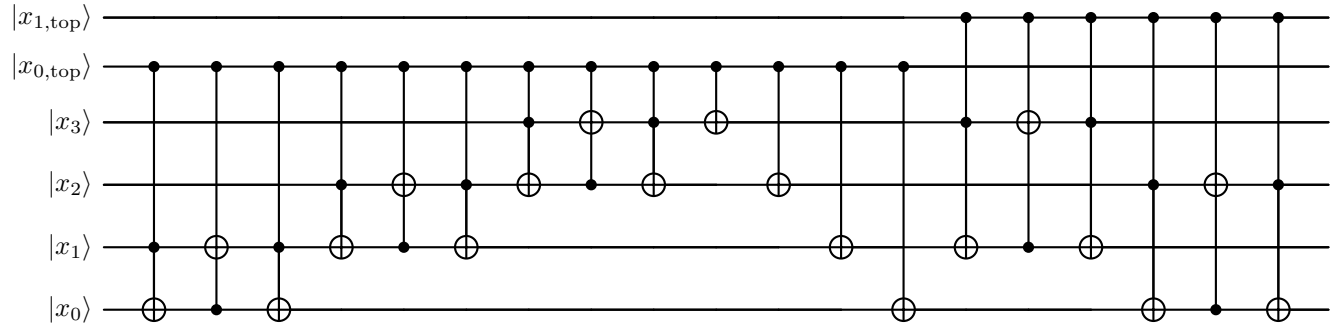

Figure 9: Combined controlled circuit applying $7x \bmod 15$ conditioned on $|x_{0,\text{top}}\rangle$ (second wire), followed by $4x \bmod 15$ conditioned on $|x_{1,\text{top}}\rangle$ (top wire). The top 6 wires are not rendered.

Measurement and period recovery. After applying the Quantum Fourier Transform and measuring the circuit multiple times, we obtain a distribution of outcomes. The peaks in this distribution correspond to integer multiples of $\frac{2^{2n}}{r}$. By selecting high-probability outcomes and applying the continued fractions algorithm, we successfully recover the period $r = 4$.

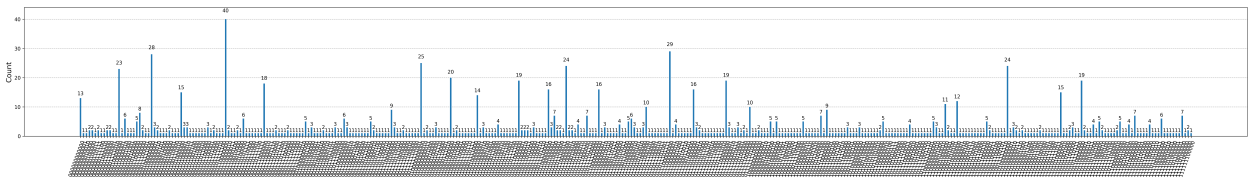

Figure 10: Measurement results after QFT

Remark. In the literature, the Quantum Fourier Transform (QFT) and its inverse are sometimes used interchangeably. While they differ by complex conjugation of phases, they yield identical measurement probabilities and therefore do not affect the outcome of the algorithm.

2 Current Progress

We now have a complete picture of how Shor's algorithm operates. Given an integer N, we select a random integer a such that $1 < a < N$. The algorithm then processes a through the quantum circuit, producing a measurement outcome $|y\rangle$. This value is used in a classical post-processing step—the continued fractions algorithm—to extract the period r of the function $a^x \bmod N$.

Once r is obtained, we verify whether it satisfies the necessary conditions: namely, that r is even and that $a^r \equiv 1 \pmod{N}$. If these conditions hold, we apply the difference-of-squares identity and compute

$$\gcd\left(a^{\frac{r}{2}} - 1, N\right), \quad \gcd\left(a^{\frac{r}{2}} + 1, N\right),$$

which, with high probability, yield nontrivial factors of N.

A subtle but crucial point is that any implementation capable of breaking RSA must support *arbitrary* inputs a and N. In other words, the circuit must be fully general: it should accept a and N as inputs and perform modular exponentiation dynamically within the computation.

However, most current implementations of Shor's algorithm, like our toy example, fall short of this ideal. In practice, quantum circuits are typically *not base-agnostic* and *not modulus-agnostic*. Instead, they are compiled for fixed values of a and N, with the modular exponentiation stage hard-coded for those specific parameters. Changing a or N is therefore not a matter of supplying new inputs; it requires redesigning or recompiling the circuit itself.

This limitation is significant. Shor's algorithm is fundamentally an algorithmic procedure, not a collection of isolated demonstrations. A generic implementation should behave analogously to a classical program: it should accept arbitrary inputs and perform the computation.

Just as one cannot claim to have built a two-digit calculator if the circuitry must be rewired to compute $2+3$ versus $2+2$, one cannot claim to have constructed a general factoring circuit for a given N if the circuit must be reconfigured for different choices of a.

2.1 Circuits for $N = 15$

As mentioned above, if we fix $N = 15$, a valid generic factoring circuit should be able to handle arbitrary choices of a that are coprime to N. That is, the circuit should accept any $a \in \{2, 4, 7, 8, 11, 13, 14\}$ as input and correctly execute the algorithm.

However, none of the current experimental realizations that claim to have "factored" 15 satisfy this requirement. In this section, we examine two prominent examples: IBM's implementation of Shor's algorithm on an NMR quantum computer [2], and the realization by Monz *et al.* on a trapped-ion quantum computer [3].

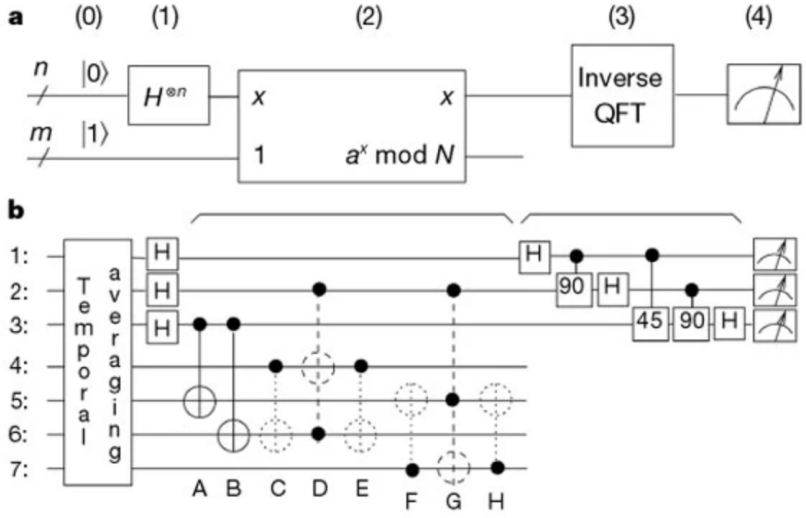

Figure 11: IBM's circuit for $a = 7$ and $N = 15$ [2]

IBM implementation. The IBM team demonstrated two cases: $a = 11$, which they refer to as the "easy" case, and $a = 7$, the "difficult" case. The distinction arises from the corresponding orders: for $a = 11$, the period is $r = 2$, requiring only a single modular multiplication, whereas for $a = 7$, the period is $r = 4$, requiring two nontrivial modular multiplications.

To reduce experimental complexity, the circuit was heavily optimized. For instance, instead of using the standard $2n = 8$ qubits in the upper register, the implementation employs only three qubits. While such optimizations make the experiment feasible, they also hard-code the structure of the circuit for specific values of a and N.

As a result, the circuit cannot accommodate arbitrary inputs a. Under our criterion, this does not constitute a general-purpose factoring circuit for $N = 15$, but rather a tailored demonstration for particular instances of a and N.

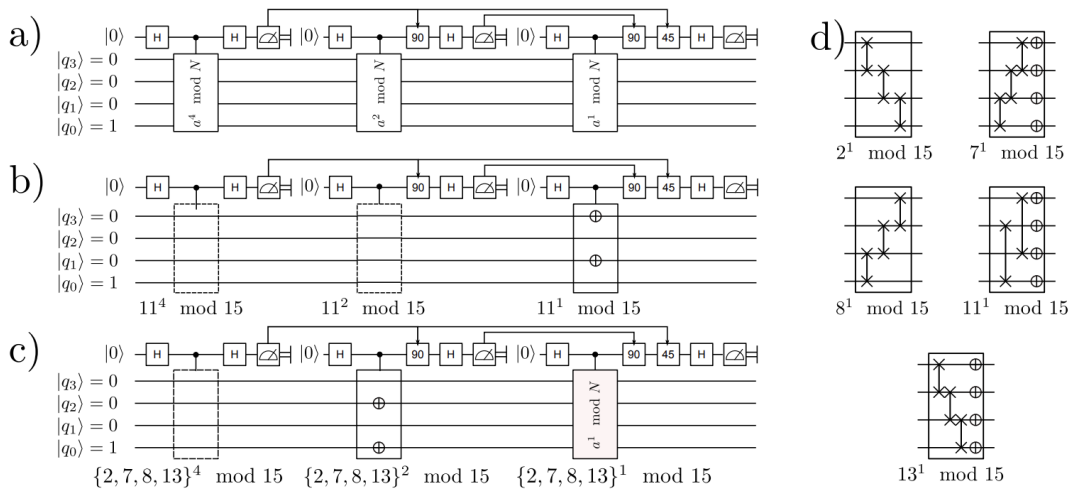

Figure 12: Monz *et al.*'s realization of Shor's algorithm [3]

Trapped-ion implementation. Monz *et al.* improved upon the IBM implementation by employing a semi-classical Quantum Fourier Transform [6], which reduces the number of qubits required in the upper register from $2n$ to just one. This represents a significant experimental advancement in terms of resource efficiency.

However, despite acknowledging the importance of constructing circuits that support arbitrary bases a, their implementation remains dependent on specific choices of a, as evidenced by the circuit structure (see Fig. 12(d) in their work). The modular exponentiation stage is still effectively hard-coded.

Discussion. Therefore, under our (admittedly stringent) criterion, existing experiments that claim to have factored 15 do not realize a truly general factoring circuit. Instead, they demonstrate specific, precompiled instances of the algorithm, rather than a scalable, input-agnostic implementation capable of handling arbitrary bases a for a fixed modulus $N = 15$.

2.2 Circuits for $N = 21$

A similar situation arises in experiments that claim to have factored 21. Consider the work of Martín-López *et al.* [4], which implements a compiled version of Shor's algorithm using a two-photon system to factor $N = 21$.

As in the trapped-ion implementation by Monz *et al.*, this approach employs qubit recycling in the upper register to reduce the number of required qubits. Furthermore, to minimize resources in the lower register, the authors fix the base to $a = 4$. With this choice, the modular exponentiation $a^x \bmod 21$ only explores three of the 2^5 possible states of a 5-qubit register [4] (which apparently eliminated any possibility of generic factoring). This observation allows them to replace the entire lower register with a single qutrit.

While this optimization is both clever and experimentally efficient, it comes at the cost of generality. The resulting circuit is explicitly tailored to the specific choice $a = 4$ and does not extend to arbitrary bases. As such, it does not constitute a general-purpose factoring circuit for $N = 21$.

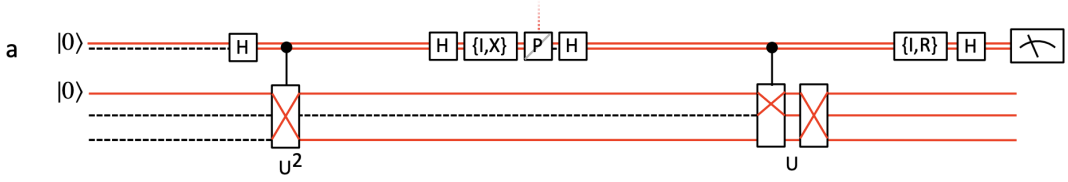

Figure 13: Martín-López *et al.*'s two-photon implementation of Shor's algorithm [4]

2.3 What Does It Take to Build a Generic Factoring Circuit?

A truly general factoring circuit should accept arbitrary inputs a and N and, without requiring any structural modification, produce the measurement outcome $|y\rangle$ needed to recover the period r.

Achieving this requires the construction of general-purpose quantum modular multiplication units, or more broadly, quantum arithmetic circuits that operate correctly for any choice of a and N. This idea is not new; the foundational principles of quantum arithmetic circuits were established in early work such as [7].

3 Quantum Arithmetic Circuits

3.1 Why Can't We Reuse Classical Arithmetic Circuits?

At first glance, one might wonder why classical arithmetic circuits cannot be directly adapted for use in quantum computation. The key obstacle is that classical circuits typically rely on *irreversible* gates, whereas quantum circuits must be constructed entirely from *reversible* operations.

For example, consider a classical AND gate. If the output is 0, it is impossible to uniquely determine the input: the input could have been $(A, B) = (0, 0)$, $(1, 0)$, or $(0, 1)$. In other words, the mapping from input to output is many-to-one, and therefore not invertible. This loss of information makes the operation irreversible.

In contrast, quantum computation requires all operations (except measurement) to be unitary. Unitary operators are inherently reversible: given the output state, one can always recover the input state by applying the inverse operation. This reversibility is not merely a technical constraint, but a fundamental requirement for preserving quantum coherence, superposition, and entanglement.

Indeed, the computational advantage of quantum systems arises precisely from these properties. For example, applying Hadamard gates allows us to prepare a superposition over 2^n input states. To meaningfully manipulate such superpositions, all intermediate operations must preserve the full quantum state, which necessitates reversibility.

Consequently, we must design quantum arithmetic circuits using reversible gate sets. Classical arithmetic circuits, in their standard form, cannot be directly reused; instead, they must be reformulated in a reversible framework suitable for quantum computation.

3.2 How to Build Quantum Arithmetic Circuits For Shor's Algorithm?

In Shor's algorithm, quantum arithmetic circuits are primarily used in the modular exponentiation stage, which, as previously discussed, can be reduced to modular multiplication. Modular multiplication can in turn be decomposed into repeated modular additions.

This follows from the identity

$$a \cdot b \bmod N = \left(\sum_{k=1}^{b} (a \bmod N) \right) \bmod N,$$

which expresses multiplication as repeated addition.

Therefore, the fundamental building block of quantum arithmetic circuits for Shor's algorithm is the *modular adder*. Modular adders themselves are constructed from standard quantum adders, combined with additional logic to enforce the modulus.

The workflow for a modular addition circuit can be summarized as follows:

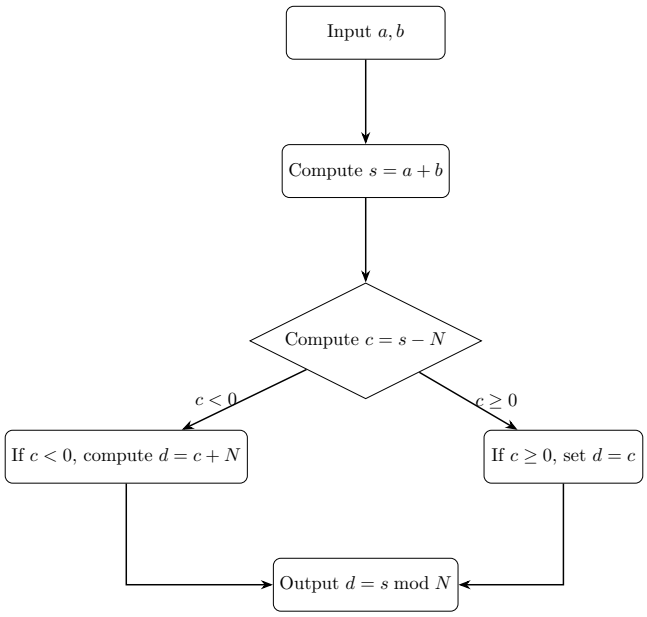

Figure 14: Workflow of modular addition

Here, the value $-N$ (in two's complement notation) can be obtained by flipping the bits of N and adding 1.

Conceptually, this shows that modular exponentiation can be decomposed into a sequence of controlled modular addition operations. However, despite this seemingly straightforward decomposition, constructing efficient quantum arithmetic circuits remains highly nontrivial.

3.3 What Is Preventing Us from Building Them?

Quantum Adders Require Significant Resources

Vedral *et al.* proposed a realization of a quantum adder using reversible gates [7], illustrated below:

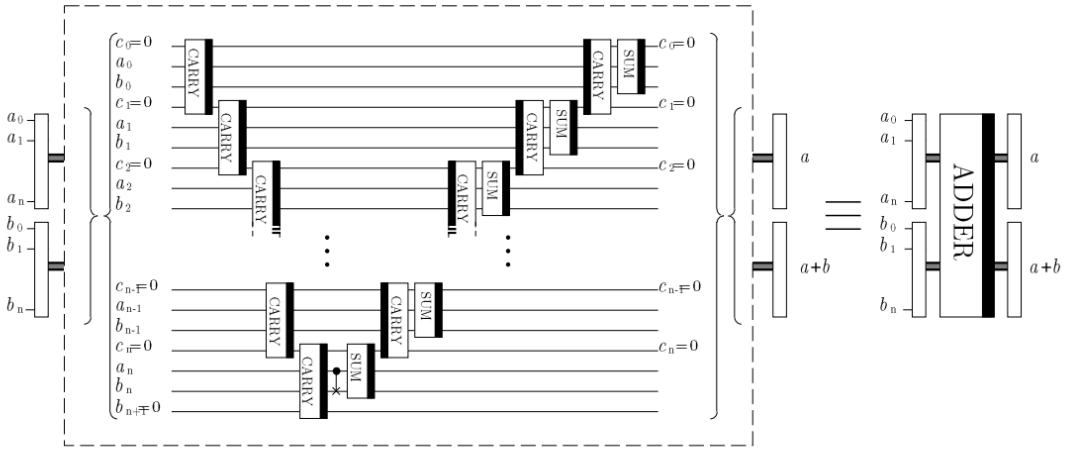

Figure 15: Quantum adder circuit [7]

One immediate observation is that this circuit is relatively deep. This arises from the use of the ripple-carry method, in which carry bits are computed sequentially and later *uncomputed* to restore ancillary qubits. This uncomputation is necessary to ensure that intermediate states do not remain entangled with the output, allowing the circuit to be reused within a larger computation.

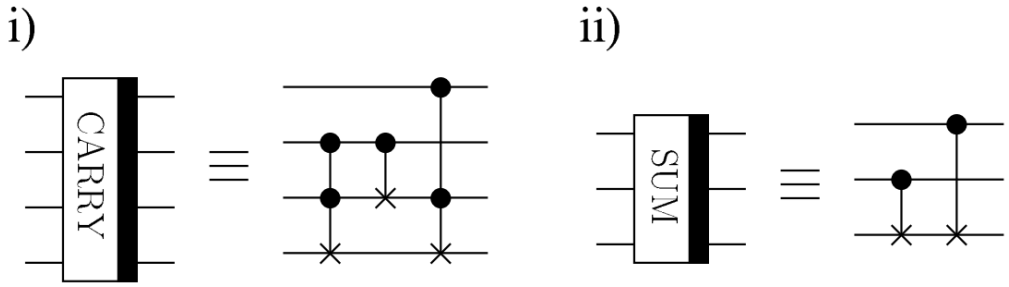

Figure 16: Breakdown of carry and sum gates [7]

In addition, the circuit is built from repeated applications of *sum* and *carry* modules, which are themselves composed of CNOT and Toffoli gates. While the number of gates per module appears deceptively modest

(e.g., two gates for a sum operation and three for a carry operation), it is important to recognize that Toffoli gates are significantly more expensive to implement than single- and two-qubit gates.

To see this more clearly, consider a typical decomposition of the Toffoli gate:

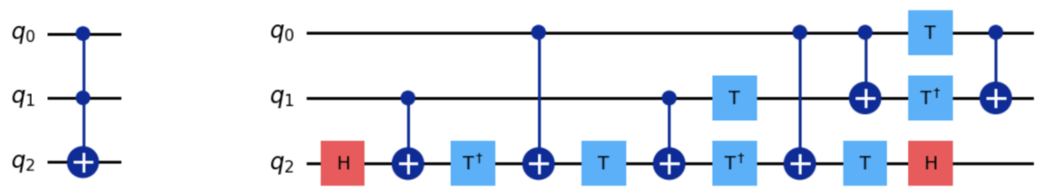

Figure 17: Decomposition of a Toffoli gate

As illustrated, a Toffoli gate is decomposed into a sequence of Clifford gates (such as CNOT and Hadamard gates) and non-Clifford gates (notably T and $T^\dagger$ gates). While Clifford gates are relatively straightforward to implement and are naturally supported by many quantum error-correcting codes, non-Clifford gates—particularly T gates—are considerably more resource-intensive.

In fault-tolerant quantum computation, T gates typically require *magic state distillation*, a process in which noisy auxiliary states are purified and then consumed to implement the desired operation. This procedure incurs substantial overhead in terms of additional qubits, circuit depth, and runtime.

Consequently, even seemingly simple arithmetic primitives such as adders become expensive at scale, as they rely heavily on Toffoli gates. This resource overhead represents a major obstacle to constructing efficient, large-scale quantum arithmetic circuits.

The Cost of General-Base, General-Modulus Factoring

Our criterion for a general factoring circuit is that it must operate without rewiring for arbitrary inputs a and N. This requirement significantly restricts the types of optimizations we can exploit. In particular, many optimizations that are valid for specific choices of a and N cannot be used in a fully general implementation.

To see this, note that for certain pairs (a, N) with small order r, many of the modular exponentiation operations become trivial. Consider the following example.

Let $a = 10^6$ and $N = 10^{12} + 1$. The modulus N is large (requiring approximately 40 bits to represent), so a general implementation would allocate $2n = 80$ qubits in the upper register and, correspondingly, require up to 80 controlled modular multiplications.

However, for this particular choice of a and N, the order is $r = 4$. This means that after the first two nontrivial modular multiplications, all subsequent operations effectively act as the identity. If we were allowed to classically precompute these values and tailor the circuit accordingly, we would only need to implement two modular multiplications.

In contrast, a general-purpose circuit cannot take advantage of this simplification. It must be capable of handling arbitrary inputs, including cases where the order r is large. As a result, all modular multiplication blocks must be implemented, even if many of them turn out to be redundant for specific inputs.

```
print([pow(1000000, 2**k, 1000000000001) for k in range(80)])
```

```
[1000000, 1000000000000, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

Figure 18: Modular exponentiation structure for $a = 10^6$ and $N = 10^{12} + 1$

This illustrates a key challenge: generality comes at a significant resource cost. A circuit that supports arbitrary inputs must include a large number of modular multiplication units, each built from many quantum adders, making scalable implementations of Shor's algorithm extremely demanding.

4 Conclusion and Outlook

Shor's algorithm demonstrates how quantum systems can leverage superposition and entanglement to solve problems with rich mathematical structure—in this case, the detection of periodicity. It stands as one of the most compelling examples of a quantum algorithm offering an exponential advantage over classical methods.

However, current experimental demonstrations that claim to have factored integers using Shor's algorithm fall short of realizing fully general implementations. In particular, they do not construct circuits that are both base-agnostic and modulus-agnostic. Instead, these experiments rely on heavily optimized, problem-specific circuits that are tailored to fixed choices of a and N.

The primary obstacle to building such general circuits lies in the resource demands of quantum arithmetic. Modular exponentiation requires a large number of quantum adders, which in turn rely on Toffoli gates and other costly operations. These gates are expensive to implement in a fault-tolerant setting, both in terms of qubit overhead and circuit depth, and remain challenging to realize with current noisy hardware.

Despite these challenges, significant progress has been made in the design of quantum arithmetic circuits [8]. Advances in circuit optimization have dramatically reduced the estimated resources required to factor large integers, with recent work lowering the qubit requirements for breaking RSA from tens of millions to on the order of one million qubits [9, 10].

Looking forward, continued improvements in qubit fabrication, gate fidelity, and quantum error correction will be essential for scaling quantum algorithms to practical regimes. As these technologies mature, more complex and fully general implementations of algorithms such as Shor's may become feasible, bringing us closer to realizing their full computational potential.

References

[1] Peter W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. *FOCS*, 1994.

[2] Lieven Vandersypen, Matthias Steffen, G. Breyta, Costantino Yannoni, and Mark Sherwood. Experimental realization of shor's quantum factoring algorithm using nuclear magnetic resonance. nature, 414:883. *Nature*, 414:883–7, 12 2001.

[3] Thomas Monz, Daniel Nigg, Esteban A. Martinez, Matthias F. Brandl, Philipp Schindler, Richard Rines, Shannon X. Wang, Isaac L. Chuang, and Rainer Blatt. Realization of a scalable shor algorithm. *Science*, 351(6277):1068–1070, 2016.

[4] Enrique Martín-López, Anthony Laing, Thomas Lawson, Roberto Alvarez, Xiao-Qi Zhou, and Jeremy L. O'Brien. Experimental realization of shor's quantum factoring algorithm using qubit recycling. *Nature Photonics*, 6(11):773–776, 2012.

[5] Leah Crane. Quantum computer sets new record for finding prime number factors. *New Scientist*, 2019. Accessed: 2026-04-25.

[6] Robert B. Griffiths and Chi-Sheng Niu. Semiclassical fourier transform for quantum computation. *Physical Review Letters*, 76(17):3228–3231, 1996.

[7] Vlatko Vedral, Adriano Barenco, and Artur Ekert. Quantum networks for elementary arithmetic operations. *Physical Review A*, 54(1):147–153, 1996.

[8] Steven A. Cuccaro, Thomas G. Draper, Samuel A. Kutin, and David Petrie Moulton. A new quantum ripple-carry addition circuit. *arXiv preprint*, 2004.

[9] Craig Gidney and Martin Ekerå. How to factor 2048 bit rsa integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, 2021.

[10] Craig Gidney. How to factor 2048 bit rsa integers with less than a million noisy qubits. *arXiv preprint*, 2025.

[11] Charles H. Bennett. The thermodynamics of computation—a review. *International Journal of Theoretical Physics*, 21(12):905–940, 1982.

5 Appendix

In this appendix, I outline several questions that arose during this project but remain only partially resolved.

Bridging Two Equivalent Interpretations of Shor's Algorithm

There are two widely accepted and equivalent ways of interpreting Shor's algorithm.

The first is the perspective presented in this paper and in Shor's original work. In this view, Hadamard gates are used to prepare a superposition over 2^n values of x, after which the modular exponentiation circuit computes $|a^x \bmod N\rangle$ in the lower register. Because the function $a^x \bmod N$ is periodic, the joint state can be regrouped according to shared function values, and the Quantum Fourier Transform (QFT) is applied to extract the period r.

The second interpretation frames Shor's algorithm as an instance of quantum phase estimation. In this approach, the lower register is prepared in an eigenstate $|\psi\rangle$ of the modular exponentiation operator U. Applying controlled powers of U generates a phase that encodes information about the period r, which is then "kicked back" onto the upper register. The inverse Quantum Fourier Transform (IQFT) is subsequently used to extract this phase information.

Since these two interpretations are mathematically equivalent, one would expect them to produce similar measurement statistics when implemented on the same circuit. However, when experimenting with both QFT and IQFT in practice, I observed noticeably different measurement histograms:

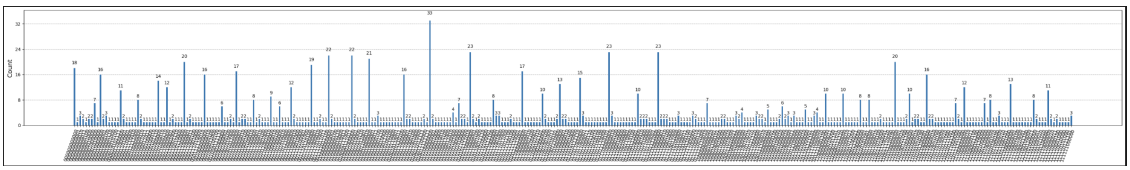

Figure 19: Measurement histogram using IQFT

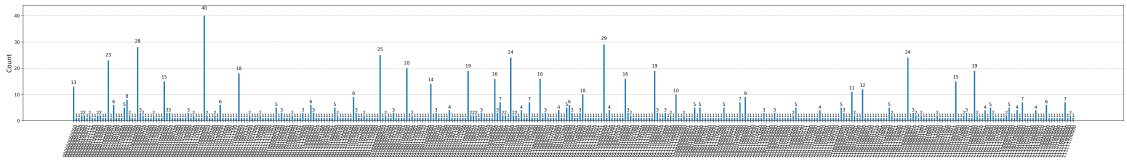

Figure 20: Measurement histogram using QFT

Although both approaches correctly recover $r = 4$, the distributions differ in appearance. The source of this discrepancy is not yet fully understood. It may arise from implementation details in the QFT library or from subtle differences in how the circuits are constructed. A natural next step would be to implement both QFT and IQFT explicitly at the gate level and compare their behavior under identical conditions.

What Counts as a Reversible Gate?

A reversible gate is typically defined as one for which the inputs can be uniquely determined from the outputs. However, this raises an interesting question.

Consider the following circuit:

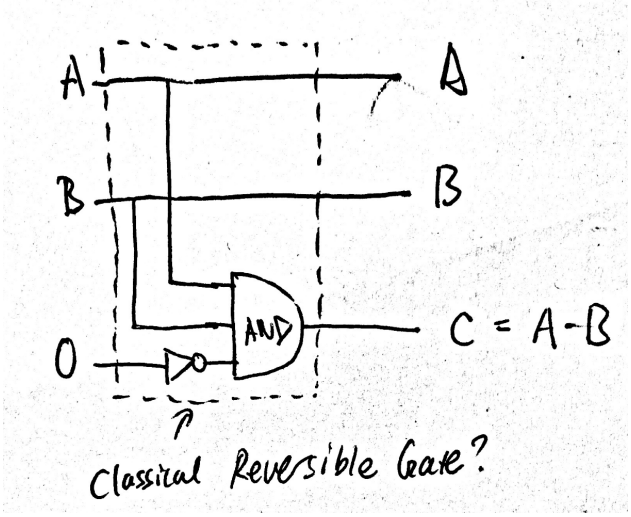

Figure 21: A candidate classical reversible gate

At first glance, the overall mapping may appear reversible, even though it incorporates an irreversible component (a 3-input AND gate that should dissipate energy [11]).

If information is locally destroyed but globally preserved, can the operation still be considered reversible? I need a better understanding of what reversibility means.

How Do Controlled Unitaries Work?

How to implement controlled unitary operations? What does the physics look like?

How Did IBM Optimize Their $a = 7, N = 15$ Circuit?

How did they reduce the number of gate count in the lower register to such a smaller number compared with our toy example circuit? What is the justification?

Why is Information Associated with Heat?

How Can We Define Information? Is its Conservation a Fundamental Law of Physics?